

# Scaling Laws for Neural Language Models

## 神经语言模型的尺度定律

Jared Kaplan · Sam McCandlish · Tom Henighan · Tom B. Brown · Benjamin Chess · Rewon Child · Scott Gray  
· Alec Radford · Jeffrey Wu · Dario Amodei  
Johns Hopkins University / OpenAI

贾里德·卡普兰 · 萨姆·麦坎德利斯 · 汤姆·赫尼甘 · 汤姆·B·布朗 · 本杰明·切斯 ·  
雷翁·柴尔德 · 斯科特·格雷 · 亚历克·拉德福德 · 杰弗里·吴 · 达里奥·阿莫代i(OpenAI 等)

*arXiv:2001.08361 [cs.LG] | Preprint, January 2020*

### Abstract

#### 摘要

We study empirical scaling laws for language model performance on the cross-entropy loss. The loss scales as a power-law with model size, dataset size, and the amount of compute used for training, with some trends spanning more than seven orders of magnitude. Other architectural details such as network width or depth have minimal effects within a wide range. Simple equations govern the dependence of overfitting on model/dataset size and the dependence of training speed on model size. These relationships allow us to determine the optimal allocation of a fixed compute budget. Larger models are significantly more sample-efficient, such that optimally compute-efficient training involves training very large models on a relatively modest amount of data and stopping significantly before convergence.

我们研究了语言模型性能(交叉熵损失)随尺度变化的实证定律。损失随模型规模、数据集规模以及训练所用的算力呈幂律关系,部分趋势跨越了超过七个数量级。在较宽的范围内,网络宽度或深度等其他架构细节的影响很小。简单的方程即可刻画过拟合对模型/数据规模的依赖,以及训练速度对模型规模的依赖。这些关系使我们能够确定在固定算力预算下的最优分配方式。更大的模型显著更加样本高效,因此,算力最优的训练方式是:用相对适量的数据去训练一个非常大的模型,并在远未收敛时就提前停止训练。

## 1. Introduction

### 引言

Language provides a natural domain for the study of artificial intelligence, as the vast majority of reasoning tasks can be efficiently expressed and evaluated in language, and the world's text provides a wealth of data for unsupervised learning via generative modeling. Deep learning has recently seen rapid progress in language modeling, with state of the art models approaching human-level performance on many specific tasks, including the composition of coherent multi-paragraph prompted text samples.

语言是研究人工智能的一个自然领域——绝大多数推理任务都能被高效地用语言表达与评估,而世界上的文本为通过生成式建模进行无监督学习提供了海量数据。近年来,深度学习在语言建模上取得了飞速进展,最先进的方法已在许多特定任务上接近人类水平,包括生成连贯的多段提示文本。

One might expect language modeling performance to depend on model architecture, the size of neural models, the computing power used to train them, and the data available for this training process. In this work we will empirically investigate the dependence of language modeling loss on all of these factors, focusing on the Transformer architecture. The high ceiling and low floor for performance on language tasks allows us to study trends over more than seven orders of magnitude in scale.

人们可能预期,语言建模的性能会依赖于模型架构、神经网络的规模、训练所用的算力,以及可用于训练的数据。在本文中,我们将实证地研究语言建模损失对所有这些因素的依赖,并聚焦于Transformer 架构。语言任务性能的高上限和低下限,使我们能够在超过七个数量级的尺度上研究其趋势。

Throughout we will observe precise power-law scalings for performance as a function of training time, context length, dataset size, model size, and compute budget.

在本文中,我们将观察到:性能作为训练时间、上下文长度、数据集规模、模型规模和算力预算的函数,呈现精确的幂律关系。

## 1.1 Summary

### 主要结论概述

Our key findings for Transformer language models are as follows.

本文关于 Transformer 语言模型的关键发现如下。

Performance depends strongly on scale, weakly on model shape. Model performance depends most strongly on scale, which consists of three factors: the number of model parameters  $N$  (excluding embeddings), the size of the dataset  $D$ , and the amount of compute  $C$  used for training. Within reasonable limits, performance depends very weakly on other architectural hyperparameters such as depth vs. width.

性能强烈依赖于尺度,微弱依赖于模型形状。模型性能最强烈地依赖于尺度,它由三个因素组成:模型参数量  $N$ (不含嵌入)、数据集规模  $D$ ,以及训练所用算力  $C$ 。在合理范围内,深度与宽度等其它架构超参数对性能的影响非常微弱。

Smooth power laws. Performance has a power-law relationship with each of the three scale factors  $N$ ,  $D$ ,  $C$  when not bottlenecked by the other two, with trends spanning more than six orders of magnitude. We observe no signs of deviation from these trends on the upper end, though performance must flatten out eventually before reaching zero loss.

平滑的幂律。在不因其它两者而成为瓶颈的前提下,性能与  $N$ 、 $D$ 、 $C$  三个尺度因子各自呈幂律关系,趋势跨越超过六个数量级。在上限附近,我们没有观察到这些趋势出现偏离的迹象,尽管性能必然会在损失为零之前趋于平缓。

Universality of overfitting. Performance improves predictably as long as we scale up  $N$  and  $D$  in tandem, but enters a regime of diminishing returns if either  $N$  or  $D$  is held fixed while the other increases. The performance penalty depends predictably on the ratio  $N^{0.74} / D$ , meaning that every time we increase the model size  $8\times$ , we only need to increase the data by roughly  $5\times$  to avoid a penalty.

过拟合的普适性。只要  $N$  与  $D$  同步增大,性能就会可预测地提升;但若其中一个固定而另一个增大,则会进入收益递减的区域。性能惩罚可由比值  $N^{0.74} / D$  可预测地刻画,这意味着:每当模型规模扩大 8 倍,我们只需将数据扩大约 5 倍即可避免过拟合惩罚。

Universality of training. Training curves follow predictable power-laws whose parameters are roughly independent of the model size. By extrapolating the early part of a training curve, we can roughly predict the loss that would be achieved if we trained for much longer.

训练过程的普适性。训练曲线遵循可预测的幂律,其参数与模型规模大致无关。通过对训练曲线早期部分进行外推,我们可以粗略预测在更长时间训练下能达到的损失。

Transfer improves with test performance. When we evaluate models on text with a different distribution than they were trained on, the results are strongly correlated to those on the training validation set with a roughly constant offset in the loss — in other words, transfer to a different distribution incurs a constant penalty but otherwise improves roughly in line with performance on the training set.

迁移随测试性能提升而提升。当我们在与训练数据分布不同的文本上评估模型时,结果与训练验证集上的结果强相关,损失的偏移大致为常数——换言之,迁移到不同分布会带来一个常数惩罚,除此之外,其提升大致与训练集上的性能同步。

Sample efficiency. Large models are more sample-efficient than small models, reaching the same level of performance with fewer optimization steps and using fewer data points.

样本效率。更大的模型比小模型更加样本高效:它们可以用更少的优化步数、更少的数据点达到相同的性能水平。

Convergence is inefficient. When working within a fixed compute budget  $C$  but without any other restrictions on the model size  $N$  or available data  $D$ , we attain optimal performance by training very large models and stopping significantly short of convergence. Maximally compute-efficient training would therefore be far more sample efficient than one might expect based on training small models to convergence, with data requirements growing very slowly as  $D \sim C^{0.27}$  with training compute.

收敛是低效的。在固定的算力预算  $C$  下,若不对模型规模  $N$  或可用数据  $D$  施加其它约束,我们应当训练非常大的模型,并在远未收敛时就提前停止训练。因此,算力最优的训练比基于训练小模型至收敛的预期要样本高效得多,数据需求随训练算力增长得非常缓慢,大致为  $D \propto C^{\{0.27\}}$ 。

Optimal batch size. The ideal batch size for training these models is roughly a power of the loss only, and continues to be determinable by measuring the gradient noise scale; it is roughly 1–2 million tokens at convergence for the largest models we can train.

最优批大小。这些模型训练的理想批大小大致仅由损失的某个幂次决定,并且仍然可以通过梯度噪声规模来确定;在我们所能训练的最大模型上,收敛时的批大小约为 100 – 200 万个标记。

Taken together, these results show that language modeling performance improves smoothly and predictably as we appropriately scale up model size, data, and compute. We expect that larger language models will perform better and be more sample efficient than current models.

综上,这些结果表明:只要我们适当增大模型规模、数据和算力,语言建模性能就会平滑且可预测地提升。我们预期,更大的语言模型将比当前模型表现更好、且更样本高效。

## 1.2 Summary of Scaling Laws

### 尺度定律汇总

The test loss of a Transformer trained to autoregressively model language can be predicted using a power-law when performance is limited by only either the number of non-embedding parameters  $N$ , the dataset size  $D$ , or the optimally allocated compute budget  $C_{\min}$ :

当性能仅受到非嵌入参数量  $N$ 、数据集规模  $D$  或最优算力预算  $C_{\min}$  其中之一的限制时,可以用幂律来预测自回归语言 Transformer 的测试损失:

$$L(N) = (N_c / N)^{\alpha_N}, \alpha_N \approx 0.076, N_c \approx 8.8 \times 10^{13} \text{ (non-embed params)}$$

$$L(D) = (D_c / D)^{\alpha_D}, \alpha_D \approx 0.095, D_c \approx 5.4 \times 10^{13} \text{ tokens}$$

$$L(C_{\min}) = (C_c^{\min} / C_{\min})^{\alpha_{C^{\min}}}, \alpha_{C^{\min}} \approx 0.050, C_c^{\min} \approx 3.1 \times 10^8 \text{ PF-days}$$

$$L(N) = (N_c / N)^{\alpha_N}, \alpha_N \approx 0.076, N_c \approx 8.8 \times 10^{13} \text{ (非嵌入参数量)}$$

$$L(D) = (D_c / D)^{\alpha_D}, \alpha_D \approx 0.095, D_c \approx 5.4 \times 10^{13} \text{ tokens}$$

$$L(C_{\min}) = (C_c^{\min} / C_{\min})^{\alpha_{C^{\min}}}, \alpha_{C^{\min}} \approx 0.050, C_c^{\min} \approx 3.1 \times 10^8 \text{ PF-days}$$

These relations hold across eight orders of magnitude in  $C_{\min}$ , six orders of magnitude in  $N$ , and over two orders of magnitude in  $D$ . They depend very weakly on model shape and other Transformer hyperparameters (depth, width, number of self-attention heads), with specific numerical values associated with the WebText2 training set. The power laws  $\alpha_N$ ,  $\alpha_D$ ,  $\alpha_{C^{\min}}$  specify the degree of performance improvement expected as we scale up  $N$ ,  $D$ , or  $C_{\min}$ ; for example, doubling the number of parameters yields a loss that is smaller by a factor  $2^{\{-\alpha_N\}} \approx 0.95$ . The precise numerical values of  $N_c$ ,  $C_c^{\min}$ , and  $D_c$  depend on the vocabulary size and tokenization and hence do not have a fundamental meaning.

这些关系在  $C_{\min}$  上跨越 8 个数量级、在  $N$  上跨越 6 个数量级、在  $D$  上跨越 2 个数量级。它们几乎与模型形状及其它 Transformer 超参数(深度、宽度、自注意力头数)无关,具体数值与 WebText2 训练集相对应。幂律指数  $\alpha_N$ 、 $\alpha_D$ 、 $\alpha_{C^{\min}}$  刻画了随  $N$ 、 $D$ 、 $C_{\min}$  扩大而带来的性能提升程度;例如,参数量翻倍,损失减小  $2^{\{-\alpha_N\}} \approx 0.95$  倍。 $N_c$ 、 $C_c^{\min}$ 、 $D_c$  的具体数值依赖于词表大小和分词方式,故并无根本性意义。

The critical batch size, which determines the speed/efficiency tradeoff for data parallelism, also roughly obeys a power law in  $L$ :  $B_{\text{crit}}(L) = B_* / L^{\{1/\alpha_B\}}$ , with  $B_* \approx 2 \times 10^8$  tokens and  $\alpha_B \approx 0.21$ .

决定数据并行速度/效率权衡的临界批大小,也大致服从关于  $L$  的幂律:  $B_{\text{crit}}(L) = B_* / L^{\{1/\alpha_B\}}$ , 其中  $B_* \approx 2 \times 10^8$  tokens,  $\alpha_B \approx 0.21$ 。

Equations (1) and (2) together suggest that as we increase the model size, we should increase the dataset size sublinearly according to  $D \propto N^{\{\alpha_N / \alpha_D\}} \sim N^{\{0.74\}}$ . In fact, we find that there is a single equation combining (1) and (2) that governs the simultaneous dependence on  $N$  and  $D$  and governs the degree of overfitting:

公式 (1) 与 (2) 共同表明:随着模型规模增大,我们应当按  $D \propto N^{\{\alpha_N / \alpha_D\}} \sim N^{\{0.74\}}$

这种次线性关系增加数据。事实上,我们找到了一个同时刻画  $N$  与  $D$  依赖关系并刻画过拟合程度的统一公式:

$$L(N, D) = [(N_c / N)^{\alpha_N / \alpha_D} + D_c / D]^{\alpha_D}$$

$$L(N, D) = [(N_c / N)^{\alpha_N / \alpha_D} + D_c / D]^{\alpha_D}$$

with fits pictured on the left in Figure 2. We conjecture that this functional form may also parameterize the trained log-likelihood for other generative modeling tasks.

其拟合结果如图 2 左侧所示。我们猜想,这一函数形式也可能刻画其它生成式建模任务中训练得到的对数似然。

When training a given model for a finite number of parameter update steps  $S$  in the infinite data limit, after an initial transient period, the learning curves can be accurately fit by  $L(N, S) = (N_c / N)^{\alpha_N} + (S_c / S_{\min}(S))^{\alpha_S}$ , where  $S_c \approx 2.1 \times 10^3$  and  $\alpha_S \approx 0.76$ , and  $S_{\min}(S)$  is the minimum possible number of optimization steps estimated using the adjusted-steps formula.

在无穷数据极限下,给定模型经有限步参数更新  $S$  训练时,经过初始的瞬态期后,学习曲线可以被  $L(N, S) = (N_c / N)^{\alpha_N} + (S_c / S_{\min}(S))^{\alpha_S}$  精确拟合,其中  $S_c \approx 2.1 \times 10^3$ ,  $\alpha_S \approx 0.76$ ,  $S_{\min}(S)$  是由调整后的步数公式估计出的最优化步数。

When training within a fixed compute budget  $C$ , but with no other constraints, the equation above leads to the prediction that the optimal model size  $N$ , optimal batch size  $B$ , optimal number of steps  $S$ , and dataset size  $D$  should grow as  $N \propto C^{\alpha_C^{\min} / \alpha_N}$ ,  $B \propto C^{\alpha_C^{\min} / \alpha_B}$ ,  $S \propto C^{\alpha_C^{\min} / \alpha_S}$ ,  $D = B \cdot S$ , with  $\alpha_C^{\min} = 1 / (1/\alpha_S + 1/\alpha_B + 1/\alpha_N)$ , which closely matches the empirically optimal results  $N \propto C_{\min}^{0.73}$ ,  $B \propto C_{\min}^{0.24}$ , and  $S \propto C_{\min}^{0.03}$ . As the computational budget  $C$  increases, it should be spent primarily on larger models, without dramatic increases in training time or dataset size.

在固定算力预算  $C$  下,若不施加其它约束,上述方程可预测最优模型规模  $N$ 、最优批大小  $B$ 、最优步数  $S$  以及数据集规模  $D$  应满足:  $N \propto C^{\alpha_C^{\min} / \alpha_N}$ ,  $B \propto C^{\alpha_C^{\min} / \alpha_B}$ ,  $S \propto C^{\alpha_C^{\min} / \alpha_S}$ ,  $D = B \cdot S$ , 其中  $\alpha_C^{\min} = 1 / (1/\alpha_S + 1/\alpha_B + 1/\alpha_N)$ 。这与经验最优结果  $N \propto C_{\min}^{0.73}$ 、 $B \propto C_{\min}^{0.24}$ 、 $S \propto C_{\min}^{0.03}$  高度吻合。随着算力预算  $C$  的增加,应主要投入到更大的模型上,而训练时间或数据集规模则不必大幅增加。

### 1.3 Notation

#### 记号说明

We use the following notation:

本文使用的记号如下:

- $L$  — the cross entropy loss in nats. Typically averaged over the tokens in a context, but in some cases we report the loss for specific tokens within the context.
- $N$  — the number of model parameters, excluding all vocabulary and positional embeddings.
- $C \approx 6 N B S$  — an estimate of the total non-embedding training compute, where  $B$  is the batch size, and  $S$  is the number of training steps (parameter updates). We quote numerical values in PF-days (1 PF-day =  $8.64 \times 10^{19}$  FLOPs).
- $D$  — the dataset size in tokens.
- $B_{\text{crit}}$  — the critical batch size, defined and discussed in Section 5.1. Training at  $B_{\text{crit}}$  provides a roughly optimal compromise between time and compute efficiency.
- $C_{\min}$  — an estimate of the minimum amount of non-embedding compute to reach a given value of the loss.
- $S_{\min}$  — an estimate of the minimal number of training steps needed to reach a given value of the loss.
- $\alpha_X$  — power-law exponents for the scaling of the loss as  $L(X) \propto 1 / X^{\alpha_X}$  where  $X$  can be any of  $N$ ,  $D$ ,  $C$ ,  $S$ ,  $B$ ,  $C_{\min}$ .

•  $L$  — 交叉熵损失(以 nats 为单位)。通常取上下文内标记上的平均值;但在某些情况下,会报告上下文中特定位置的损失。

•  $N$  — 模型参数数量,不包括所有词嵌入与位置嵌入。

•  $C \approx 6 N B S$  — 非嵌入训练算力的估计值,其中  $B$  为批大小,  $S$  为训练步数(即参数更新次数)。我们以 PF-days 为单位给出数值(1 PF-day =  $8.64 \times 10^{19}$  次浮点运算)。

•  $D$  — 数据集规模(以 token 为单位)。

•  $B_{\text{crit}}$  — 临界批大小,定义与讨论见第 5.1 节。在  $B_{\text{crit}}$  下训练大致能在时间与算力效率之间取得最优折中。

•  $C_{\min}$  — 达到给定损失值所需的最小非嵌入算力的估计。

•  $S_{\min}$  — 达到给定损失值所需的最少训练步数的估计。

- $\alpha_X$  —— 损失尺度关系的幂律指数,即  $L(X) \propto 1/X^{\alpha_X}$ ,其中  $X$  可以是  $N$ 、 $D$ 、 $C$ 、 $S$ 、 $B$ 、 $C_{\min}$  中的任意一个。

## 2. Background and Methods

### 背景与方法

We train language models on WebText2, an extended version of the WebText dataset, tokenized using byte-pair encoding with a vocabulary size  $n_{\text{vocab}} = 50257$ . We optimize the autoregressive log-likelihood (i.e. cross-entropy loss) averaged over a 1024-token context, which is also our principal performance metric. We record the loss on the WebText2 test distribution and on a selection of other text distributions. We primarily train decoder-only Transformer models, though we also train LSTM models and Universal Transformers for comparison.

我们在 WebText2(扩展版 WebText 数据集)上训练语言模型,使用字节对编码 (BPE) 进行分词,词表大小  $n_{\text{vocab}} = 50257$ 。我们优化 1024 token 上下文上的自回归对数似然(即交叉熵损失),这也是我们主要的性能指标。我们在 WebText2 测试集分布以及若干其它文本分布上记录损失。我们主要训练仅含解码器的 Transformer 模型,同时为对比也训练了 LSTM 模型与 Universal Transformer。

### 2.1 Parameter and Compute Scaling of Transformers

#### Transformer 的参数与算力尺度

We parameterize the Transformer architecture using hyperparameters  $n_{\text{layer}}$  (number of layers),  $d_{\text{model}}$  (dimension of the residual stream),  $d_{\text{ff}}$  (dimension of the intermediate feed-forward layer),  $d_{\text{attn}}$  (dimension of the attention output), and  $n_{\text{heads}}$  (number of attention heads per layer). We include  $n_{\text{ctx}}$  tokens in the input context, with  $n_{\text{ctx}} = 1024$  except where otherwise noted.

我们用以下超参数刻画 Transformer

架构: $n_{\text{layer}}$ (层数)、 $d_{\text{model}}$ (残差流维度)、 $d_{\text{ff}}$ (中间前馈层维度)、 $d_{\text{attn}}$ (注意力输出维度)以及  $n_{\text{heads}}$ (每层注意力头数)。输入上下文包含  $n_{\text{ctx}}$  个 token,除特别说明外, $n_{\text{ctx}} = 1024$ 。

We use  $N$  to denote the model size, which we define as the number of non-embedding parameters:  $N \approx 2 d_{\text{model}} n_{\text{layer}} (2 d_{\text{attn}} + d_{\text{ff}}) = 12 n_{\text{layer}} d_{\text{model}}^2$  with the standard  $d_{\text{attn}} = d_{\text{ff}} / 4 = d_{\text{model}}$ , where we have excluded biases and other sub-leading terms. Our models also have  $n_{\text{vocab}} \cdot d_{\text{model}}$  parameters in an embedding matrix, and use  $n_{\text{ctx}} \cdot d_{\text{model}}$  parameters for positional embeddings, but we do not include these when discussing the model size  $N$ ; we will see that this produces significantly cleaner scaling laws.

我们用  $N$  表示模型规模,定义其为非嵌入参数量:  $N \approx 2 d_{\text{model}} n_{\text{layer}} (2 d_{\text{attn}} + d_{\text{ff}}) = 12 n_{\text{layer}} d_{\text{model}}^2$  (在标准设置  $d_{\text{attn}} = d_{\text{ff}} / 4 = d_{\text{model}}$  下),其中已排除偏置及其它次要项。模型还有  $n_{\text{vocab}} \cdot d_{\text{model}}$  个词嵌入矩阵参数,以及  $n_{\text{ctx}} \cdot d_{\text{model}}$  个位置嵌入参数;但在讨论模型规模  $N$  时,我们不计入这些参数——之后我们会看到,这样做能带来显著更干净的尺度定律。

Evaluating a forward pass of the Transformer involves roughly  $C_{\text{forward}} \approx 2 N + 2 n_{\text{layer}} n_{\text{ctx}} d_{\text{model}}$  add-multiply operations, where the factor of two comes from the multiply-accumulate operation used in matrix multiplication. A more detailed per-operation parameter and compute count is included in Table 1. For contexts and models with  $d_{\text{model}} > n_{\text{ctx}} / 12$ , the context-dependent computational cost per token is a relatively small fraction of the total compute. Since we primarily study models where  $d_{\text{model}} \gg n_{\text{ctx}} / 12$ , we do not include context-dependent terms in our training compute estimate. Accounting for the backwards pass (approximately twice the compute as the forwards pass), we then define the estimated non-embedding compute as  $C \approx 6 N$  floating point operators per training token.

评估 Transformer 的一次前向传播大约需要  $C_{\text{forward}} \approx 2 N + 2 n_{\text{layer}} n_{\text{ctx}} d_{\text{model}}$  次乘加运算(其中因子 2 来自矩阵乘法中的乘累加操作)。更详细的每操作参数与算力计数见表 1。对于  $d_{\text{model}} > n_{\text{ctx}} / 12$  的上下文与模型,与上下文相关的每 token 计算成本在总算力中只占很小一部分。由于我们主要研究  $d_{\text{model}}$  远大于  $n_{\text{ctx}} / 12$  的模

型,因此在训练算力的估计中不计与上下文相关的项。考虑反向传播(算力约为前向传播的两倍),我们将估计的非嵌入算力定义为每训练 token 约  $C \approx 6 N$  次浮点运算。

| Operation / ■■ | Parameters / ■■■■                                            | FLOPs per Token / ■ token FLOPs |
|----------------|--------------------------------------------------------------|---------------------------------|
| Embed          | $(n_{\text{vocab}} + n_{\text{ctx}}) \cdot d_{\text{model}}$ | $4 d_{\text{model}}$            |



|                       |                                                                               |                                                                            |
|-----------------------|-------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Attention: QKV        | $n_{\text{layer}} \cdot d_{\text{model}} \cdot 3 d_{\text{attn}}$             | $2 n_{\text{layer}} \cdot d_{\text{model}} \cdot 3 d_{\text{attn}}$        |
| Attention: Mask       | —                                                                             | $2 n_{\text{layer}} \cdot n_{\text{ctx}} \cdot d_{\text{attn}}$            |
| Attention: Project    | $n_{\text{layer}} \cdot d_{\text{attn}} \cdot d_{\text{model}}$               | $2 n_{\text{layer}} \cdot d_{\text{attn}} \cdot d_{\text{model}}$          |
| Feedforward           | $n_{\text{layer}} \cdot 2 d_{\text{model}} \cdot d_{\text{ff}}$               | $2 n_{\text{layer}} \cdot 2 d_{\text{model}} \cdot d_{\text{ff}}$          |
| De-embed              | —                                                                             | $2 d_{\text{model}} \cdot n_{\text{vocab}}$                                |
| Total (Non-Embedding) | $N = 2 d_{\text{model}} n_{\text{layer}} (2 d_{\text{attn}} + d_{\text{ff}})$ | $C_{\text{fwd}} = 2 N + 2 n_{\text{layer}} n_{\text{ctx}} d_{\text{attn}}$ |

Table 1 — Parameter counts and compute (forward pass) estimates for a Transformer model. Sub-leading terms such as nonlinearities, biases, and layer normalization are omitted.

1 — Transformer

## 2.2 Training Procedures

### 训练流程

Unless otherwise noted, we train models with the Adam optimizer for a fixed  $2.5 \times 10^5$  steps with a batch size of 512 sequences of 1024 tokens. Due to memory constraints, our largest models (more than 1B parameters) were trained with Adafactor. We experimented with a variety of learning rates and schedules, as discussed in the Appendix. We found that results at convergence were largely independent of learning rate schedule. Unless otherwise noted, all training runs included in our data used a learning rate schedule with a 3000 step linear warmup followed by a cosine decay to zero.

除非另有说明,我们使用 Adam 优化器进行固定  $2.5 \times 10^5$  步的训练,批大小为 512 个长度为 1024 token 的序列。受显存限制,我们最大的模型(超过 10 亿参数)使用 Adafactor 训练。我们尝试了多种学习率与学习率调度(详见附录)。我们发现,收敛结果几乎与学习率调度无关。除非另有说明,数据中包含的所有训练运行均使用 3000 步线性 warmup 之后接余弦衰减至零的学习率调度。

## 2.3 Datasets

### 数据集

We train our models on an extended version of the WebText dataset. The original WebText dataset was a web scrape of outbound links from Reddit through December 2017 which received at least 3 karma. In WebText2, we added outbound Reddit links from January to October 2018, also with a minimum of 3 karma. The karma threshold served as a heuristic for whether people found the link interesting or useful. The text of the new links was extracted with the Newspaper3k python library. In total, the dataset consists of 20.3M documents containing 96 GB of text and  $1.62 \times 10^{11}$  words. We then apply the reversible tokenizer, which yields  $2.29 \times 10^{11}$  tokens. We reserve  $6.6 \times 10^8$  of these tokens for use as a test set, and we also test on similarly-prepared samples of Books Corpus, Common Crawl, English Wikipedia, and a collection of publicly-available Internet Books.

我们在 WebText 数据集的扩展版上训练模型。原始 WebText 是从 Reddit 中抽取的外链网页抓取,截至 2017 年 12 月,要求每条链接至少获得 3 个 karma。在 WebText2 中,我们新增了 2018 年 1 月至 10 月的 Reddit 外链,同样要求至少 3 个 karma。karma 阈值作为一条启发式规则,用以判断链接是否对人们有趣或有用。新增链接的文本通过 Newspaper3k 这个 Python 库抽取。整体而言,数据集共包含 2030 万篇文档、96 GB 文本、 $1.62 \times 10^{11}$  个词。然后我们应用可逆的分词器,得到  $2.29 \times 10^{11}$  个 token。我们将其中  $6.6 \times 10^8$  个 token 留作测试集,同时还在 Books Corpus、Common Crawl、英文维基百科以及一组公开可用的 Internet Books 的同种处理样本上进行测试。

## 3. Empirical Results and Basic Power Laws

### 实验结果与基本幂律

To characterize language model scaling we train a wide variety of models, varying a number of factors including: model size (ranging in size from 768 to 1.5 billion non-embedding parameters); dataset size (ranging from 22 million to 23 billion tokens); shape (including depth, width, attention heads, and feed-forward dimension); context length (1024 for most runs, though we also experiment with shorter contexts); and batch size ( $2^{11}$  for most runs, but we also vary it to measure the critical batch size).

为刻画语言模型的尺度,我们在多种因素上变化训练:模型规模(从 768 到 15 亿非嵌入参数);数据集规模(从 200 万到 230 亿 token);形状(深度、宽度、注意力头、前馈维度);上下文长度(大多数运行使用 1024,同时也实验了更短的上下文);批大小(大多数运行使用  $2^{19}$ ,但我们也改变它以测量临界批大小)。

In this section we will display data along with empirically-motivated fits, deferring theoretical analysis to later sections. 本节我们将展示数据及由经验驱动的拟合,而将理论分析留到后续章节。

### 3.1 Approximate Transformer Shape and Hyperparameter Independence

Transformer 形状与超参数近似无关性

Transformer performance depends very weakly on the shape parameters  $n_{\text{layer}}$ ,  $n_{\text{heads}}$ , and  $d_{\text{ff}}$  when we hold the total non-embedding parameter count  $N$  fixed. To establish these results we trained models with fixed size while varying a single hyperparameter. This was simplest for the case of  $n_{\text{heads}}$ . When varying  $n_{\text{layer}}$ , we simultaneously varied  $d_{\text{model}}$  while keeping  $N \approx 12 n_{\text{layer}} d_{\text{model}}^2$  fixed. Similarly, to vary  $d_{\text{ff}}$  at fixed model size we also simultaneously varied the  $d_{\text{model}}$  parameter, as required by the parameter counts in Table 1. Independence of  $n_{\text{layers}}$  would follow if deeper Transformers effectively behave as ensembles of shallower models, as has been suggested for ResNets.

在保持总非嵌入参数量  $N$  不变的情况下,Transformer 的性能几乎不依赖于形状超参数  $n_{\text{layer}}$ 、 $n_{\text{heads}}$  和  $d_{\text{ff}}$ 。为建立这一结论,我们在固定规模的同时,逐个改变某一个超参数进行了实验。最直接的是  $n_{\text{heads}}$ 。改变  $n_{\text{layer}}$  时,我们同时调整  $d_{\text{model}}$  以保持  $N \approx 12 n_{\text{layer}} d_{\text{model}}^2$  不变。类似地,在固定  $N$  下改变  $d_{\text{ff}}$  时,我们也同时改变  $d_{\text{model}}$ (如表 1 中的参数量公式所要求的)。如果更深的 Transformer 实际上表现为若干浅层模型的集成(有人曾对 ResNet 提出过类似观点),那么  $n_{\text{layer}}$  的无关性也就顺理成章了。

### 3.2 Performance with Non-Embedding Parameter Count $N$

性能与非嵌入参数量  $N$  的关系

We display the performance of a wide variety of models, ranging from small models with shape  $(n_{\text{layer}}, d_{\text{model}}) = (2, 128)$  through billion-parameter models, ranging in shape from  $(6, 4288)$  through  $(207, 768)$ . Here we have trained to near convergence on the full WebText2 dataset and observe no overfitting (except possibly for the very largest models).

我们展示了大量模型的性能——从形状  $(n_{\text{layer}}, d_{\text{model}}) = (2, 128)$  的小模型,到形状从  $(6, 4288)$  到  $(207, 768)$  的十亿参数模型。我们是在完整的 WebText2 数据集上训练至接近收敛的,且没有观察到过拟合(最大的几个模型可能除外)。

We find a steady trend with non-embedding parameter count  $N$ , which can be fit to the first term of the  $L(N, D)$  equation, so that  $L(N) \approx (N_c / N)^{\alpha_N}$ . To observe these trends it is crucial to study performance as a function of  $N$ ; if we instead use the total parameter count (including the embedding parameters) the trend is somewhat obscured. This suggests that the embedding matrix can be made smaller without impacting performance, as has been seen in recent work.

我们发现非嵌入参数量  $N$  与性能呈现稳定的趋势,可以由  $L(N, D)$  公式的第一项拟合,即  $L(N) \approx (N_c / N)^{\alpha_N}$ 。要观察这一趋势,必须以  $N$  为自变量来研究性能;若使用总参数量(含嵌入参数),趋势会变得模糊。这说明词嵌入矩阵可以做得更小而不影响性能——近期工作中也观察到了这一点。

Although these models have been trained on the WebText2 dataset, their test loss on a variety of other datasets is also a power-law in  $N$  with nearly identical power, as shown in Figure on generalization vs model size.

尽管这些模型只在 WebText2 上训练,但它们在多种其它数据集上的测试损失同样与  $N$  呈幂律关系,且幂律指数几乎相同(参见泛化与模型规模图)。

### 3.3 Performance with Dataset Size and Compute

性能与数据集规模、算力的关系

We display empirical trends for the test loss as a function of dataset size  $D$  (in tokens) and training compute  $C$  in Figure on basic power laws. For the trend with  $D$  we trained a model with  $(n_{\text{layer}}, n_{\text{embd}}) = (36, 1280)$  on fixed subsets of the WebText2 dataset. We stopped training once the test loss ceased to decrease. We see that the resulting test losses can be fit with simple power-law  $L(D) \approx (D_c / D)^{\alpha_D}$  in the dataset size.

我们在基本幂律图中展示了测试损失作为数据集规模  $D$ (以 token 计)和训练算力  $C$  的函数所呈现的经验趋势。在与  $D$  的关系上,我们在 WebText2 的固定子集上训练了一个形状为  $(n_{\text{layer}}, n_{\text{embd}}) = (36, 1280)$  的模型,并在测试损失不再下降时停止训练。我们发现,所得的测试损失可由  $L(D) \approx$

$(D_c / D)^{\alpha_D}$  这一简单的幂律拟合。

The total amount of non-embedding compute used during training can be estimated as  $C = 6 N B S$ , where  $B$  is the batch size,  $S$  is the number of parameter updates, and the factor of 6 accounts for the forward and backward passes. Thus for a given value of  $C$  we can scan over all models with various  $N$  to find the model with the best performance on step  $S = C / (6 B S)$ . Note that in these results the batch size  $B$  remains fixed for all models, which means that these empirical results are not truly optimal. We will account for this in later sections using an adjusted  $C_{\min}$  to produce cleaner trends. The result can be fit with  $L(C) \approx (C_c / C)^{\alpha_C}$ . The data strongly suggests that sample efficiency improves with model size.

训练过程中使用的总非嵌入算力可估计为  $C = 6 N B S$ , 其中  $B$  为批大小,  $S$  为参数更新次数, 因子 6 涵盖了前向与反向传播。因此, 对于给定的  $C$ , 我们可以在不同  $N$  的模型上扫描, 找到在  $S = C / (6 B S)$  步上性能最佳的模型。需要注意的是, 在这些结果中, 所有模型使用的批大小  $B$  都是固定的, 这意味着这些经验结果并非真正最优。我们将在后续章节中通过调整后的  $C_{\min}$  来解释这一点, 从而得到更干净的趋势。结果可由  $L(C) \approx (C_c / C)^{\alpha_C}$  拟合。数据强烈表明: 样本效率随模型规模的增大而提升。

## 4. Charting the Infinite Data Limit and Overfitting

### 刻画无穷数据极限与过拟合

In Section 3 we found a number of basic scaling laws for language modeling performance. Here we will study the performance of a model of size  $N$  trained on a dataset with  $D$  tokens while varying  $N$  and  $D$  simultaneously. We will empirically demonstrate that the optimally trained test loss accords with the scaling law  $L(N, D)$ . This provides guidance on how much data we would need to train models of increasing size while keeping overfitting under control.

在第 3 节中, 我们发现了若干基本的语言建模性能尺度定律。本节中, 我们将研究在同时变化  $N$  与  $D$  时, 一个规模为  $N$ 、在  $D$  个 token 数据集上训练的模型的性能。我们将以经验方式证明: 经过最优训练的测试损失服从  $L(N, D)$  这一尺度定律。这为我们在控制过拟合的前提下训练不断增大的模型所需的数据量提供了指引。

### 4.1 Proposed $L(N, D)$ Equation

#### 提出的 $L(N, D)$ 公式

We have chosen the parameterization  $L(N, D) = [(N_c / N)^{\alpha_N / \alpha_D} + D_c / D]^{\alpha_D}$  using three principles:

我们采用  $L(N, D) = [(N_c / N)^{\alpha_N / \alpha_D} + D_c / D]^{\alpha_D}$  这一参数形式, 基于三条原则:

- Changes in vocabulary size or tokenization are expected to rescale the loss by an overall factor. The parameterization of  $L(N, D)$  (and all models of the loss) must naturally allow for such a rescaling.
- Fixing  $D$  and sending  $N \rightarrow \infty$ , the overall loss should approach  $L(D)$ . Conversely, fixing  $N$  and sending  $D \rightarrow \infty$  the loss must approach  $L(N)$ .
- $L(N, D)$  should be analytic at  $D = \infty$ , so that it has a series expansion in  $1/D$  with integer powers. Theoretical support for this principle is significantly weaker than for the first two.
  - 词表大小或分词方式的变化预期只会对损失乘以一个整体常数。因此,  $L(N, D)$  (以及所有刻画损失的模型) 的参数形式必须自然地允许这种整体缩放。
  - 固定  $D$  并令  $N \rightarrow \infty$ , 整体损失应趋于  $L(D)$ 。反过来, 固定  $N$  并令  $D \rightarrow \infty$ , 损失必须趋于  $L(N)$ 。
  - $L(N, D)$  应当在  $D = \infty$  处解析, 从而在  $1/D$  上可做整数次幂的级数展开。这一原则的理论支持显著弱于前两条。

Our choice of  $L(N, D)$  satisfies the first requirement because we can rescale  $N_c, D_c$  with changes in the vocabulary. This also implies that the values of  $N_c, D_c$  have no fundamental meaning. Since we stop training early when the test loss ceases to improve and optimize all models in the same way, we expect that larger models should always perform better than smaller models. But with fixed finite  $D$ , we also do not expect any model to be capable of approaching the best possible loss (i.e. the entropy of text). Similarly, a model with fixed size will be capacity-limited. These considerations motivate our second principle. Knowledge of  $L(N)$  at infinite  $D$  and  $L(D)$  at infinite  $N$  fully determines all the parameters in  $L(N, D)$ . The third principle explains the asymmetry between the roles of  $N$  and  $D$  in the equation. Very similar symmetric expressions are possible, but they would not have a  $1/D$  expansion with integer powers, and would require the introduction of an additional parameter. In any case, we will see that our equation for  $L(N, D)$  fits the data well, which is the most important justification for our ansatz.



我们选择的  $L(N, D)$  满足第一条要求,因为我们可以通过词表变化对  $N_c$ 、 $D_c$  进行缩放。这也意味着  $N_c$ 、 $D_c$  的具体数值并无根本意义。由于我们在测试损失不再下降时提前停止训练,并以相同方式优化所有模型,我们预期更大的模型总是优于更小的模型;但在固定有限  $D$  下,我们不预期任何模型能达到可能的最佳损失(即文本的熵)。类似地,固定规模的模型存在容量上限。这些考量是第二条原则的依据。在无穷  $D$  处的  $L(N)$  与在无穷  $N$  处的  $L(D)$  完全确定了  $L(N, D)$  中的所有参数。第三条原则解释了公式中  $N$  与  $D$  之间的不对称性。类似的、形式上更对称的表达式是存在的,但它们在  $1/D$  上没有整数次幂的展开,且需要引入额外的参数。无论如何,我们将看到, $L(N, D)$  公式能很好地拟合数据,这是我们采用这一假设的最重要依据。

## 4.2 Results

### 实验结果

We regularize all our models with 10% dropout, and by tracking test loss and stopping once it is no longer decreasing. The results are displayed in Figure 3, including a fit to the four parameters  $\alpha_N$ ,  $\alpha_D$ ,  $N_c$ ,  $D_c$  in  $L(N, D)$ :  $\alpha_N = 0.076$ ,  $\alpha_D = 0.103$ ,  $N_c = 6.4 \times 10^{13}$ ,  $D_c = 1.8 \times 10^{13}$ .

我们对所有模型使用 10% 的 dropout,并通过追踪测试损失、待其不再下降时停止训练。拟合  $L(N, D)$  中四个参数  $\alpha_N$ 、 $\alpha_D$ 、 $N_c$ 、 $D_c$  的结果如图 3 所示:  $\alpha_N = 0.076$ ,  $\alpha_D = 0.103$ ,  $N_c = 6.4 \times 10^{13}$ ,  $D_c = 1.8 \times 10^{13}$ 。

We obtain an excellent fit, with the exception of the runs where the dataset has been reduced by a factor of 1024, to about  $2 \times 10^7$  tokens. With such a small dataset, an epoch consists of only 40 parameter updates. Perhaps such a tiny dataset represents a different regime for language modeling, as overfitting happens very early in training.

除了将数据集缩减为原来的  $1/1024$ (约  $2 \times 10^7$  token)的那些运行外,我们都得到了优秀的拟合。这么小的数据集,一个 epoch 仅包含 40 次参数更新。或许这种极小的数据集代表语言建模的一种不同模式,因为过拟合在训练初期就会发生。

To chart the borderlands of the infinite data limit, we can directly study the extent of overfitting. For all but the largest models, we see no sign of overfitting when training with the full 22B token WebText2 dataset, so we can take it as representative of  $D = \infty$ . Thus we can compare finite  $D$  to the infinite data limit by defining  $\delta L(N, D) \equiv L(N, D) / L(N, \infty) - 1$  and studying it as a function of  $N$ ,  $D$ . We see empirically that  $\delta L$  depends only a specific combination of  $N$  and  $D$ , as shown in the appendix.

为了刻画无穷数据极限的边界,我们可以直接研究过拟合的程度。除最大的模型之外,在完整 220 亿 token 的 WebText2 数据集上训练时,我们看不到任何过拟合迹象,因此可以把它视为  $D = \infty$  的代表。我们可以定义  $\delta L(N, D) \equiv L(N, D) / L(N, \infty) - 1$ ,以此来比较有限  $D$  与无穷数据极限。经验上我们发现,  $\delta L$  只依赖于  $N$  与  $D$  的某个特定组合(详见附录)。

We estimate that the variation in the loss with different random seeds is roughly 0.02, which means that to avoid overfitting when training to within that threshold of convergence we require  $D \gtrsim (5 \times 10^3) \cdot N^{0.74}$ . With this relation, models smaller than  $10^{10}$  parameters can be trained with minimal overfitting on the 22B token WebText2 dataset, but our largest models will encounter some mild overfitting. More generally, this relation shows that dataset size may grow sub-linearly in model size while avoiding overfitting. Note however that this does not typically represent maximally compute-efficient training. We should also emphasize that we have not optimized regularization (e.g. the dropout probability) while varying dataset and model size.

我们估计,在不同随机种子下损失的变化约为 0.02,因此为了在训练至与收敛相差此阈值时避免过拟合,我们需要  $D \gtrsim (5 \times 10^3) \cdot N^{0.74}$ 。按此关系,在 220 亿 token 的 WebText2 上训练  $10^9$  参数以下的模型几乎不会出现过拟合,但我们的最大模型会遭遇一定程度的轻微过拟合。更一般地,这一关系表明:为避免过拟合,数据集规模可以按模型规模的次线性增长。不过,这种关系通常并不代表算力最优的训练方式。我们还须强调,在变化数据集与模型规模时,我们并未优化正则化(如 dropout 概率)。

## 5. Scaling Laws with Model Size and Training Time

### 模型规模与训练时间的尺度定律

In this section we will demonstrate that a simple scaling law provides a good description for the loss as a function of model size  $N$  and training time. First we will explain how to use previous results to define a universal training step  $S_{\min}$ , which accounts for the fact that most of our models have not been trained at an optimal batch size. Then we will demonstrate that we can fit the model size and training time dependence of the loss using  $L(N, S)$ . Later we will use these results to predict the optimal allocation of training compute between model size and training time, and then confirm that prediction.

本节中,我们将证明:一个简单的尺度定律能够很好地刻画损失作为模型规模  $N$  与训练时间的函数。首先,我们将说明如何利用已有结果定义一个普适的训练步数  $S_{\min}$ ,以解释大多数模型并未在最优批大小下训练这一事实。然后,我们将证明可以用  $L(N, S)$  来拟合损失对模型规模与训练时间的依赖。后续我们将利用这些结果预测模型规模与训练时间之间的最优算力分配,并通过实验加以验证。

## 5.1 Adjustment for Training at $B_{\text{crit}}(L)$

### 按 $B_{\text{crit}}(L)$ 调整训练

A simple empirical theory for the batch size dependence of training was developed in prior work. It was argued that there is a critical batch size  $B_{\text{crit}}$  for training; for  $B$  up to  $B_{\text{crit}}$  the batch size can be increased with very minimal degradation in compute-efficiency, whereas for  $B > B_{\text{crit}}$  increases in  $B$  result in diminishing returns. It was also argued that the gradient noise scale provides a simple prediction for  $B_{\text{crit}}$ , and that neither depends directly on model size except through the value of the loss that has been attained. These results can be used to predict how training time and compute will vary with the batch size. To utilize both training time and compute as effectively as possible, it is best to train with a batch size  $B \approx B_{\text{crit}}$ .

前人工作发展了一套关于训练中批大小依赖的简单经验理论。该理论认为训练存在一个 临界批大小  $B_{\text{crit}}$ :在  $B \leq B_{\text{crit}}$  的范围内,增大批大小对算力效率的影响极小;而当  $B > B_{\text{crit}}$  时,继续增大  $B$  会出现收益递减。该理论还指出,梯度噪声规模为  $B_{\text{crit}}$  提供了一种简单的预测,且两者都只通过损失值间接依赖于模型规模,而不直接依赖于  $N$ 。这些结果可以预测训练时间与算力随批大小的变化。要同时最有效地利用时间与算力,最好在  $B \approx B_{\text{crit}}$  下进行训练。

More specifically, it was demonstrated that for a wide variety of neural network tasks, the number of training steps  $S$  and the number of data examples processed  $E = BS$  satisfy the simple relation  $(S / S_{\min} - 1)(E / E_{\min} - 1) = 1$  when training to any fixed value of the loss  $L$ . Here  $S_{\min}$  is the minimum number of steps necessary to reach  $L$ , while  $E_{\min}$  is the minimum number of data examples that must be processed. This relation defines the critical batch size  $B_{\text{crit}}(L) \equiv E_{\min} / S_{\min}$ , which is a function of the target value of the loss. Training at the critical batch size makes a roughly optimal time/compute tradeoff, requiring  $2 S_{\min}$  training steps and processing  $E = 2 E_{\min}$  data examples.

更具体地说,前人证明:对于多种神经网络任务,在训练至任意固定的损失值  $L$  时,训练步数  $S$  与处理的数据样本数  $E = BS$  满足简单关系  $(S / S_{\min} - 1)(E / E_{\min} - 1) = 1$ 。其中  $S_{\min}$  是达到  $L$  所需的最少步数,  $E_{\min}$  是必须处理的最少样本数。该关系定义了临界批大小  $B_{\text{crit}}(L) \equiv E_{\min} / S_{\min}$ ,它只是目标损失值的函数。在临界批大小下训练大致能给出一个最优的时间/算力折中:需要  $2 S_{\min}$  步训练,处理  $E = 2 E_{\min}$  个样本。

In our experiments we plotted the critical batch size and gradient noise scale as a function of training loss for two different models. We see that  $B_{\text{crit}}(L)$  is independent of model size, and only depends on the loss  $L$ . So the predictions continue to hold for Transformer language models. The critical batch size can be fit with a power-law in the loss  $B_{\text{crit}}(L) \approx B_* / L^{1/\alpha_B}$ , where  $B_* \approx 2 \times 10^4$  and  $\alpha_B \approx 0.21$ . We have chosen this parameterization for  $B_{\text{crit}}(L)$  because as the loss approaches its minimum value  $L_{\min}$ , the gradient noise scale is expected to diverge, and we expect  $B_{\text{crit}}$  to track this noise scale.

在我们的实验中,我们针对两个不同的模型,绘制了临界批大小与梯度噪声规模随训练损失的变化曲线。我们发现,  $B_{\text{crit}}(L)$  与模型规模无关,只依赖于损失  $L$ 。因此,这些预测同样适用于 Transformer 语言模型。临界批大小可由关于损失的幂律拟合:  $B_{\text{crit}}(L) \approx B_* / L^{1/\alpha_B}$ , 其中  $B_* \approx 2 \times 10^4$ ,  $\alpha_B \approx 0.21$ 。我们之所以选择这种参数形式,是因为当损失接近其最小值  $L_{\min}$  时,梯度噪声规模预期会发散,我们预期  $B_{\text{crit}}$  会跟踪这一噪声规模。

We will use  $B_{\text{crit}}(L)$  to estimate the relation between the number of training steps  $S$  while training at batch size  $B = 2^i$  tokens and the number of training steps while training at  $B \gg B_{\text{crit}}$ . This is simply  $S_{\min}(S) \equiv S / (1 + B_{\text{crit}}(L) / B)$  (minimum steps, at  $B \gg B_{\text{crit}}$ ) for any given target value  $L$  for the loss. This also defines a critical value of the compute needed to train to  $L$  with a model of size  $N$  if we were to train at  $B \ll B_{\text{crit}}(L)$ :  $C_{\min}(C) \equiv C / (1 + B / B_{\text{crit}}(L))$

(minimum compute, at  $B \ll B_{\text{crit}}$ ), where  $C = 6 N B S$  estimates the (non-embedding) compute used at batch size  $B$ .

我们将利用  $B_{\text{crit}}(L)$  来估计在批大小  $B = 2^{19}$  tokens 下训练所需的步数  $S$  与在  $B \gg B_{\text{crit}}$

下训练所需步数之间的关系。对任意目标损失  $L$ , 这简化为:  $S_{\text{min}}(S) \equiv S / (1 + B_{\text{crit}}(L) / B)$  (在  $B \gg B_{\text{crit}}$

下的最少步数)。这也定义了  $B \ll B_{\text{crit}}(L)$  下训练规模为  $N$  的模型至损失  $L$  所需的算力临界值:  $C_{\text{min}}(C)$

$\equiv C / (1 + B / B_{\text{crit}}(L))$  (在  $B \ll B_{\text{crit}}$  下的最少算力), 其中  $C = 6 N B S$  是在批大小  $B$

下使用的(非嵌入)算力的估计。

## 5.2 Results for $L(N, S_{\text{min}})$

$L(N, S_{\text{min}})$  的结果

Now we will use  $S_{\text{min}}$  defined above to obtain a simple and universal fit for the dependence of the loss on model size and training time in the infinite data limit. We will fit the stable, Adam-optimized training runs using  $L(N, S_{\text{min}}) = (N_{\text{c}} / N)^{\alpha_N} + (S_{\text{c}} / S_{\text{min}})^{\alpha_S}$  for the loss. We include all training steps after the warmup period of the learning rate schedule, and find a fit to the data with the parameters:  $\alpha_N = 0.077$ ,  $\alpha_S = 0.76$ ,  $N_{\text{c}} = 6.5 \times 10^{13}$ ,  $S_{\text{c}} = 2.1 \times 10^3$ .

现在我们将使用上文定义的  $S_{\text{min}}$ , 在无穷数据极限下得到一个简单而普适的、刻画损

失对模型规模与训练时间依赖的拟合。我们用  $L(N, S_{\text{min}}) = (N_{\text{c}} / N)^{\alpha_N} + (S_{\text{c}} / S_{\text{min}})^{\alpha_S}$

对稳定的、由 Adam 优化的训练运行进行拟合。我们只纳入学习率调度 warmup

之后的所有训练步, 并得到参数拟合结果:  $\alpha_N = 0.077$ ,  $\alpha_S \approx 0.76$ ,  $N_{\text{c}} = 6.5 \times 10^{13}$ ,  $S_{\text{c}} = 2.1 \times 10^3$ 。

With these parameters, we obtain the learning curve fits. Though the fits are imperfect, we believe they are quite compelling given the simplicity of the equation. The power-law dependence of the loss on  $S_{\text{min}}$  reflects the interplay of optimizer dynamics and the loss landscape. Since the fits are best late in training, when the loss may be approximately quadratic, the power-law should provide information about the spectrum of the Hessian of the loss. Its universality suggests that the Hessian eigenvalue density is roughly independent of model size.

在这些参数下, 我们得到了学习曲线的拟合结果。尽管拟合并非完美, 但考虑到该方程

的简洁性, 我们认为拟合结果相当有说服力。损失对  $S_{\text{min}}$  的幂律依赖反映了优化器

动力学与损失景观之间的相互作用。由于拟合在训练后期(此时损失近似二次时)效果

最好, 这一幂律应当能提供关于损失 Hessian 谱的信息。其普适性表明 Hessian 特征

值密度大致与模型规模无关。

## 6. Optimal Allocation of the Compute Budget

算力预算的最优分配

We displayed the empirical trend of performance as a function of the computation used during training in Figure 1. However, this result involved training at a fixed batch size  $B$ , whereas we know that in fact we could train more efficiently by training at the batch size  $B_{\text{crit}}$  discussed above. Large and small values of the loss could have been achieved with fewer samples or fewer steps, respectively, and correcting for this inefficiency by standardizing to the critical batch size results in cleaner and more predictable trends.

我们曾在图 1 中展示性能作为训练所用算力的函数的经验趋势。然而, 这一结果是在固定批大小  $B$

下训练得到的, 而事实上我们可以通过采用上文讨论的  $B_{\text{crit}}$  来更高效地训

练。更大的损失值本来可以用更少的样本来达到, 更小的损失值本来可以用更少的步数来

达到; 通过标准化到临界批大小来修正这种低效, 可以得到更干净、更可预测的趋势。

In this section we will adjust for this oversight. More importantly, we will use the results of Section 5 to determine the optimal allocation of compute between model size  $N$  and the quantity of data processed during training, namely  $2 B_{\text{crit}} S_{\text{min}}$ . We will determine this allocation both empirically and theoretically, by using the equation for  $L(N, S_{\text{min}})$ , and we will demonstrate that these methods agree.

本节中我们将修正这一疏漏。更重要的是, 我们将利用第 5 节的结果, 确定模型规模  $N$

与训练期间所处理的数据量(即  $2 B_{\text{crit}} S_{\text{min}}$ )之间的算力最优分配。我们既会从经验

上, 也会从理论上(借助  $L(N, S_{\text{min}})$  方程)确定这一分配, 并证明两种方法是吻合的。

### 6.1 Optimal Performance and Allocations

最优性能与分配

Let us first study the loss as a function of the optimally allocated compute from the adjusted compute formula. We see that as compared to the compute plot of Figure 1, the new fit with  $C_{\text{min}}$  is somewhat improved.

首先,我们研究损失作为由调整后算力公式得到的最优分配算力的函数。我们发现,与图 1 中的算力曲线相比,使用  $C_{\min}$  的新拟合略有改善。

Given  $L(C_{\min})$ , it is natural to ask for the optimal model size  $N(C_{\min})$  that provides the minimal loss with a given quantity of training compute. The optimal model size can be fit very well with a power-law  $N(C_{\min}) \propto C_{\min}^{0.73}$ . We show the effect of training models of sub-optimal sizes in the appendix.

有了  $L(C_{\min})$ ,很自然地会问:在给定训练算力下,能达到最小损失的最优模型规模  $N(C_{\min})$  是多少?最优模型规模可由幂律  $N(C_{\min}) \propto C_{\min}^{0.73}$  非常好地拟合。  
训练次优尺寸模型的效果见附录。

By definition  $C_{\min} \equiv 6 N B_{\text{crit}} S$ , and so we can use  $N(C_{\min})$  to extract further results. In particular, since prior fits show  $B \propto L^{-4.8}$  and  $L \propto C_{\min}^{-0.05}$ , we can conclude that  $B_{\text{crit}} \propto C_{\min}^{0.24}$ . This leads us to conclude that the optimal number of steps will only grow very slowly with compute, as  $S_{\min} \propto C_{\min}^{0.03}$ , matching the empirical results. In fact the measured exponent is sufficiently small that our results may even be consistent with an exponent of zero.

根据定义  $C_{\min} \equiv 6 N B_{\text{crit}} S$ ,因此我们可以利用  $N(C_{\min})$  推导出进一步的结果。  
特别地,鉴于前文的拟合给出  $B \propto L^{-4.8}$  和  $L \propto C_{\min}^{-0.05}$ ,我们可以得到  $B_{\text{crit}} \propto C_{\min}^{0.24}$ 。由此得出最优步数只会随算力增长得非常缓慢: $S_{\min} \propto C_{\min}^{0.03}$ ,这与经验结果吻合。事实上,测得的指数已经小到可能与零指数一致。

Thus we conclude that as we scale up language modeling with an optimal allocation of computation, we should predominantly increase the model size  $N$ , while simultaneously scaling up the batch size via  $B \propto B_{\text{crit}}$  with negligible increase in the number of serial steps. Since compute-efficient training uses relatively few optimization steps, additional work on speeding up early training dynamics may be warranted.

因此我们得出结论:在以算力最优分配方式扩展语言建模时,我们应主要增大模型规模  $N$ ,同时通过  $B \propto B_{\text{crit}}$  增大批大小,而串行步数几乎不必增加。由于算力高效的训练使用的优化步数相对较少,进一步研究如何加速训练早期动力学可能是值得的。

## 6.2 Predictions from $L(N, S_{\min})$

由  $L(N, S_{\min})$  给出的预测

The results for  $L(C_{\min})$  and the allocations can be predicted from the  $L(N, S_{\min})$  equation obtained in Section 5. Given our equation for  $L(N, S_{\min})$ , we can substitute  $S_{\min} = C_{\min} / (6 N B)$  and then find the minimum of the loss as a function of  $N$ , while fixing the training compute. For the loss as a function of training compute, we predict that  $L(C_{\min}) = (C_{\text{c}}^{\min} / C_{\min})^{\alpha_{\text{C}}^{\min}}$ , where  $\alpha_{\text{C}}^{\min} \equiv 1 / (1/\alpha_S + 1/\alpha_B + 1/\alpha_N) \approx 0.054$ , in excellent agreement with the exponent of Figure 4. We also predict that  $N(C_{\min}) \propto C_{\min}^{\alpha_{\text{C}}^{\min} / \alpha_N} \approx C_{\min}^{0.71}$ , which also matches the scaling of Figure on optimal model size to within a few percent. Our scaling laws provide a predictive framework for the performance of language modeling.

$L(C_{\min})$  及其分配的结果可由第 5 节得到的  $L(N, S_{\min})$  方程预测。给定  $L(N, S_{\min})$  表达式,我们可以代入  $S_{\min} = C_{\min} / (6 N B)$ ,然后在固定训练算力下求损失关于  $N$  的极小值。对损失作为训练算力的函数,我们预测为  $L(C_{\min}) = (C_{\text{c}}^{\min} / C_{\min})^{\alpha_{\text{C}}^{\min}}$ ,其中  $\alpha_{\text{C}}^{\min} \equiv 1 / (1/\alpha_S + 1/\alpha_B + 1/\alpha_N) \approx 0.054$ ,与图 4 中得到的指数吻合得极好。我们还预测  $N(C_{\min}) \propto C_{\min}^{\alpha_{\text{C}}^{\min} / \alpha_N} \approx C_{\min}^{0.71}$ ,这与最优模型规模图中显示的尺度在百分之几内一致。本文给出的尺度定律为语言建模性能提供了一个可预测的框架。

## 6.3 Contradictions and a Conjecture

矛盾与猜想

We observe no signs of deviation from straight power-law trends at large values of compute, data, or model size. Our trends must eventually level off, though, since natural language has non-zero entropy. Indeed, the trends for compute-efficient training described in this section already contain an apparent contradiction. At scales several orders of magnitude above those documented here, the performance predicted by the  $L(C_{\min})$  scaling law decreases below what should be possible given the slow growth in training data with compute.

在大算力、大数据或大模型处,我们没有观察到直线幂律趋势出现偏离的迹象。然而,这些趋势最终必然趋于平缓,因为自然语言具有非零的熵。事实上,本节所述的算力高效训练趋势已经包含了一个明显的矛盾:在比本文研究范围高几个数量级的尺度上,  $L(C_{\min})$  尺度律所预测的性能会降低到与训练数据随算力缓慢增长所能支持的最低水平之下。



Since the amount of data used by compute-efficient training grows slowly with the compute budget, the performance predicted by  $L(C_{\min})$  eventually hits a lower bound set by the  $L(D)$  power law. To keep overfitting under control, the results of Section 4 imply that we should scale the dataset size as  $D \propto N^{\{0.74\}} \propto C_{\min}^{\{0.54\}}$  where we have used the compute-efficient  $N(C_{\min})$ .

由于算力高效训练所使用的数据量随算力预算增长缓慢,  $L(C_{\min})$  所预测的性能最终会触及由  $L(D)$  幂律设定的下界。为控制过拟合, 第 4 节的结果表明数据规模应按  $D \propto N^{\{0.74\}} \propto C_{\min}^{\{0.54\}}$  增长(其中使用了算力高效的  $N(C_{\min})$ )。

Let us compare this to the data requirements of compute-efficient training. If we train at the critical batch size (i.e.  $C = 2 C_{\min}$ ) and never re-use data during training, we find that data usage grows with compute as  $D(C_{\min}) \approx (4 \times 10^1 \blacksquare \text{ tokens}) \cdot (C_{\min} / \text{PF-Day})^{\{0.26\}}$ . This is the maximum rate at which the dataset size can productively grow with compute, since it means that we are only training for a single epoch. But it grows the dataset much more slowly than in the overfitting relation. It appears to imply that compute-efficient training will eventually run into a problem with overfitting, even if the training process never re-uses any data!

让我们将其与算力高效训练的数据需求作比较。如果我们在临界批大小下训练(即  $C = 2 C_{\min}$ ), 且训练过程中不重复使用数据, 则可发现数据用量随算力的增长为  $D(C_{\min}) \approx (4 \times 10^1 \blacksquare \text{ tokens}) \cdot (C_{\min} / \text{PF-Day})^{\{0.26\}}$ 。这是数据集规模能够有效地随算力增长的最大速率, 因为这意味着我们仅对数据训练一个 epoch。但其增速仍远慢于前面给出的过拟合关系。这似乎意味着, 即便训练过程从不重复使用数据, 算力高效的训练最终也会遇到过拟合的问题。

According to our basic power laws, we expect that when we are bottlenecked by the dataset size (i.e. by overfitting), the loss should scale as  $L(D) \propto D^{\{-0.095\}}$ . This implies that the loss would scale with compute as  $L(D(C_{\min})) \propto C_{\min}^{\{-0.03\}}$  once we are data-limited. Once again, we have a contradiction, as this will eventually intersect with our prediction for  $L(C_{\min})$ , where we found a scaling  $L(C_{\min}) \propto C_{\min}^{\{-0.050\}}$ . The intersection point of  $L(D(C_{\min}))$  and  $L(C_{\min})$  occurs at  $C^* \sim 10 \blacksquare \text{ PF-Days}$ ,  $N^* \sim 10^{12}$  parameters,  $D^* \sim 10^{12}$  tokens,  $L^* \sim 1.7$  nats/token, though the numerical values are highly uncertain, varying by an order or magnitude in either direction depending on the precise values of the exponents.

根据基本幂律, 我们预期当训练被数据规模(即过拟合)所限制时, 损失应按  $L(D) \propto D^{\{-0.095\}}$  缩放。这意味着在数据受限情形下, 损失随算力的缩放为  $L(D(C_{\min})) \propto C_{\min}^{\{-0.03\}}$ 。我们再次得到一个矛盾: 它最终会与  $L(C_{\min})$  的预测( $L(C_{\min}) \propto C_{\min}^{\{-0.050\}}$ ) 相交。  $L(D(C_{\min}))$  与  $L(C_{\min})$  的交点出现在  $C^* \approx 10^4 \text{ PF-Days}$ 、 $N^* \approx 10^{12}$  参数、 $D^* \approx 10^{12}$  tokens、 $L^* \approx 1.7$  nats/token 处; 但具体数值高度不确定, 其数量级可能因指数取值不同而上下波动一个量级。

The most obvious interpretation is that our scaling laws break down at or before we reach this point, which is still many orders of magnitude away in both compute and model size. One might also conjecture that this intersection point has a deeper meaning. If we cannot increase the model size beyond  $N^*$  without qualitatively different data requirements, perhaps this means that once we reach  $C_{\min}^*$  and  $N^*$ , we have extracted all of the reliable information available in natural language data. In this interpretation,  $L^*$  would provide a rough estimate for the entropy-per-token of natural language. In this scenario, we would expect the loss trend to level off at or before  $L^*$ .

最直接的解释是: 本文的尺度定律会在到达这一交点之前(或恰好在交点处)失效——而该交点在算力和模型规模上都还距离我们许多个数量级。另一种猜想是: 这个交点具有更深层的含义。若我们不能在不对数据要求产生质的改变的前提下把模型规模扩大到  $N^*$  之外, 那么这或许意味着, 一旦我们达到  $C_{\min}^*$  与  $N^*$ , 我们就已提取出了自然语言数据中所有可被可靠获取的信息。按照这种解读,  $L^*$  将提供自然语言每 token 熵的一个粗略估计。在此情境下, 我们会预期损失趋势在  $L^*$  处或之前趋于平缓。

## 7. Related Work

### 相关工作

Power laws can arise from a wide variety of sources. Power-law scalings with model and dataset size in density estimation and in random forest models may be connected with our results. These models suggest that power-law exponents may have a very rough interpretation as the inverse of the number of relevant features in the data. Some early work found power-law scalings between performance and dataset size. More recent work also investigated scaling between model size and data

size; their work is perhaps the closest to ours in the literature. Note, however, that one of them found super-linear scaling of dataset size with model size, whereas we find a sub-linear scaling.

幂律可以来自多种多样的来源。密度估计以及随机森林模型中模型与数据集规模的幂律缩放,可能与本文结果相关。这些模型表明,幂律指数可以被粗略地理解为数据中相关特征数的倒数。早期一些工作发现了性能与数据集规模之间的幂律缩放;近期的一些工作研究了模型规模与数据规模之间的尺度关系,这可能是文献中与我们最接近的工作。不过,需要指出的是,其中一项工作发现数据集规模相对模型规模呈超线性缩放,而我们发现的是次线性缩放。

There are some parallels between our findings on optimal allocation of compute and other recent work, including power-law learning curves. EfficientNets also appear to obey an approximate power-law relation between accuracy and model size. Very recent work studies scaling with both dataset size and model size for a variety of datasets, and fits an ansatz similar to ours. EfficientNet advocates scaling depth and width exponentially (with different coefficients) for optimal performance of image models, resulting in a power-law scaling of width as a function of depth. We find that for language models this power should be roughly one when scaling up (as width/depth should remain fixed). But more importantly, we find that the precise architectural hyperparameters are unimportant compared to the overall scale of the language model. In prior work it was argued that deep models can function as ensembles of shallower models, which could potentially explain this finding.

我们在算力最优分配方面的发现与近期其它工作(包括幂律学习曲线)存在一定的相似性。EfficientNet 似乎也服从一个关于准确率与模型规模的近似幂律关系。非常近期的工作针对多种数据集研究了模型与数据规模的联合缩放,并拟合了一个与我们类似的假设。EfficientNet 主张对图像模型以指数方式(不同系数)同时缩放深度与宽度,从而得到宽度作为深度函数的幂律关系。我们发现,对于语言模型,在扩展时这个幂指数应大致为 1(即宽度/深度保持不变)。但更重要的是,我们发现精确的架构超参数与语言模型的总尺度相比并不重要。此前有工作认为,深度模型可以看作是若干浅层模型的集成,这或许能解释本文的发现。

Some studies fix computation per data example, which tends to scale in proportion to the number of model parameters, whereas we investigate scaling with both model size and the quantity of training computation. Various works have investigated generalization in highly overparameterized models, finding a “jamming transition” when the model size reaches the dataset size (this may require training many orders of magnitude beyond typical practice, and in particular does not use early stopping). We do not observe such a transition, and find that the necessary training data scales sublinearly in the model size. Expansions in the model size, particularly at large width, may provide a useful framework for thinking about some of our scaling relations. Our results on optimization, such as the shape of learning curves, can likely be explained using a noisy quadratic model, which can provide quite accurate predictions in realistic settings. Making this connection quantitative will require a characterization of the Hessian spectrum.

一些研究固定每个数据样本的计算量,其规模往往与模型参数量成正比;而我们同时考察了模型规模与训练计算量两者的缩放。已有若干工作研究了高度过参数化模型的泛化,发现当模型规模达到数据集规模时会出现堵塞相变(jamming transition)(这可能要比通常做法多若干数量级的训练步,且并未使用提前停止)。我们并未观察到这一相变,并发现所需的训练数据按模型规模呈次线性增长。模型规模上的展开,特别是大宽度下的展开,可能为理解我们的某些尺度关系提供一个有用的框架。我们关于优化的结果(例如学习曲线的形状)很可能可以用带噪二次模型来解释,该模型能在现实设置下给出相当准确的预测。把这层联系变得定量,需要对 Hessian 谱进行刻画。

## 8. Discussion

### 讨论

We have observed consistent scalings of language model log-likelihood loss with non-embedding parameter count  $N$ , dataset size  $D$ , and optimized training computation  $C_{\min}$ , as encapsulated in the  $L(N, D)$  and  $L(N, S)$  equations. Conversely, we find very weak dependence on many architectural and optimization hyperparameters. Since scalings with  $N$ ,  $D$ ,  $C_{\min}$  are power-laws, there are diminishing returns with increasing scale.

我们已经观察到语言模型对数似然损失与非嵌入参数量  $N$ 、数据集规模  $D$  以及优化后的训练算力  $C_{\min}$  之间存在一致的尺度关系,这些关系被  $L(N, D)$  与  $L(N, S)$  公式所概括。反过来,我们发现性能对许多架构与优化超参数的依赖非常弱。由于  $N$ 、 $D$ 、 $C_{\min}$

的缩放都是幂律,因此随着尺度增大,回报会递减。

We were able to precisely model the dependence of the loss on  $N$  and  $D$ , and alternatively on  $N$  and  $S$ , when these parameters are varied simultaneously. We used these relations to derive the compute scaling, magnitude of overfitting, early stopping step, and data requirements when training large language models. So our scaling relations go beyond mere observation to provide a predictive framework. One might interpret these relations as analogues of the ideal gas law, which relates the macroscopic properties of a gas in a universal way, independent of most of the details of its microscopic constituents.

我们能够精确刻画同时变化  $N$  与  $D$ (以及  $N$  与  $S$ )时损失对它们的依赖。我们利用这些关系,推导了训练大语言模型时的算力缩放、过拟合程度、提前停止的步数以及数据需求。因此,本文给出的尺度关系不只是简单的观察,而是提供了一个可预测的框架。我们可以把这种关系类比为理想气体定律:它以一种普适的方式刻画气体的宏观性质,与其微观组分的多数细节无关。

It is natural to conjecture that the scaling relations will apply to other generative modeling tasks with a maximum likelihood loss, and perhaps in other settings as well. To this purpose, it will be interesting to test these relations on other domains, such as images, audio, and video models, and perhaps also for random network distillation. At this point we do not know which of our results depend on the structure of natural language data, and which are universal. It would also be exciting to find a theoretical framework from which the scaling relations can be derived: a “statistical mechanics” underlying the “thermodynamics” we have observed. Such a theory might make it possible to derive other more precise predictions, and provide a systematic understanding of the limitations of the scaling laws.

很自然地猜想:这些尺度关系也将适用于其它以最大似然损失进行训练的生成式建模任务,甚至可能适用于更广泛的场景。为此,将这些关系在图像、音频、视频等其它领域(以及随机网络蒸馏)上进行检验是很有意义的。目前,我们尚不清楚本文的哪些结果依赖于自然语言数据的结构,哪些具有普适性。同样令人振奋的是,若能找到一个理论框架,从这个框架中推导出这些尺度关系——也就是为本文观察到的热力学提供底层的统计力学——那就更好了。这样的理论也许能让我们推导出其它更精确的预测,并对我们的尺度定律的局限性提供系统性的理解。

In the domain of natural language, it will be important to investigate whether continued improvement on the loss translates into improvement on relevant language tasks. Smooth quantitative change can mask major qualitative improvements: “more is different”. For example, the smooth aggregate growth of the economy provides no indication of the specific technological developments that underwrite it. Similarly, the smooth improvements in language model loss may hide seemingly qualitative changes in capability.

在自然语言领域,值得研究的问题是:损失上的持续改进是否能转化为相关语言任务上的改进。平滑的、定量的变化可能掩盖重大的、定性的变化:多即不同。例如,经济总量的平滑增长无法预示背后支撑它的具体技术进步;类似地,语言模型损失的平滑提升也可能掩盖看似质的能力变化。

Our results strongly suggest that larger models will continue to perform better, and will also be much more sample efficient than has been previously appreciated. Big models may be more important than big data. In this context, further investigation into model parallelism is warranted. Deep models can be trained using pipelining, which splits parameters depth-wise between devices, but eventually requires increased batch sizes as more devices are used. Wide networks on the other hand are more amenable to parallelization, since large layers can be split between multiple workers with less serial dependency. Sparsity or branching (e.g. Inception) may allow for even faster training of large networks through increased model parallelism. And using methods that grow networks as they train, it might be possible to remain on the compute-efficient frontier for an entire training run.

本文的结果强烈表明:更大的模型将继续表现更好,而且比此前所认识到的更加样本高效。大模型可能比大数据更重要。在此背景下,对模型并行的进一步研究是必要的。深度模型可以采用流水(pipelining)方式训练——将参数按深度切分到不同设备上,但在使用更多设备时最终仍需要增大batch大小。宽网络则更易于并行化,因为大型层可以被拆分到多个worker上,且串行依赖更少。稀疏化或分支结构(如 Inception)或许能通过更高的模型并行度,让大网络的训练更快。而采用让网络在训练过程中生长的方法,有可能在整轮训练中始终保持在算力高效的前沿上。

## Acknowledgements

## 致谢

We would like to thank Shan Carter, Paul Christiano, Jack Clark, Ajeya Cotra, Ethan Dyer, Jason Eisner, Danny Hernandez, Jacob Hilton, Brice Menard, Chris Olah, and Ilya Sutskever for discussions and for feedback on drafts of this work.

我们感谢 Shan Carter、Paul Christiano、Jack Clark、Ajeya Cotra、Ethan Dyer、Jason Eisner、Danny Hernandez、Jacob Hilton、Brice Menard、Chris Olah 与 Ilya Sutskever 在本文撰写过程中提供的讨论与对初稿的反馈。

## Appendix A. Summary of Power Laws

### 附录 A. 幂律汇总

For easier reference, we provide a summary of the key trends described throughout the paper.

为便于查阅,本节汇总了文中所述的关键趋势。

| Parameters / ■■ | Data / ■■ | Compute / ■■ | Batch Size / ■■■ | Equation / ■■                                                       |
|-----------------|-----------|--------------|------------------|---------------------------------------------------------------------|
| N               | $\infty$  | $\infty$     | Fixed            | $L(N) = (N_c / N)^{\alpha_N}$                                       |
| $\infty$        | D         | Early stop   | Fixed            | $L(D) = (D_c / D)^{\alpha_D}$                                       |
| Optimal         | $\infty$  | C            | Fixed            | $L(C) = (C_c / C)^{\alpha_C}$ (naive)                               |
| N_opt           | D_opt     | C_min        | $B \ll B_{crit}$ | $L(C_{min}) = (C_c^{min} / C_{min})^{\alpha_{C^{min}}}$             |
| N               | D         | Early stop   | Fixed            | $L(N, D) = [(N_c / N)^{\alpha_N / \alpha_D} + D_c / D]^{\alpha_D}$  |
| N               | $\infty$  | S steps      | $B \ll B_{crit}$ | $L(N, S) = (N_c / N)^{\alpha_N} + (S_c / S_{min}(S, B))^{\alpha_S}$ |

Table 2 — Key trend equations (summary).

■ 2 — ■■■■■■■■■■

The empirical fitted values for these trends are:

这些趋势的经验拟合值为:

| Power Law / ■■             | Exponent / ■■ | Scale (tokenization-dependent) / ■■(■■■■)     |
|----------------------------|---------------|-----------------------------------------------|
| $\alpha_N = 0.076$         | —             | $N_c = 8.8 \times 10^{13}$ params (non-embed) |
| $\alpha_D = 0.095$         | —             | $D_c = 5.4 \times 10^{13}$ tokens             |
| $\alpha_C = 0.057$         | —             | $C_c = 1.6 \times 10^4$ PF-days               |
| $\alpha_{C^{min}} = 0.050$ | —             | $C_c^{min} = 3.1 \times 10^4$ PF-days         |
| $\alpha_B = 0.21$          | —             | $B_* = 2.1 \times 10^4$ tokens                |
| $\alpha_S = 0.76$          | —             | $S_c = 2.1 \times 10^3$ steps                 |

Table 3 — Key parameters to trend fits.

■ 3 — ■■■■■■■■■■

The optimal parameters for compute efficient training are given by:

算力高效训练的最优参数为:

| Compute-Efficient Value / ■■■■■                                   | Power Law / ■■ | Scale / ■■                     |
|-------------------------------------------------------------------|----------------|--------------------------------|
| $N_{opt} = N_e \cdot C_{min}^{p_N}$                               | $p_N = 0.73$   | $N_e = 1.3 \times 10^4$ params |
| $B \ll B_{crit} = B_* / L^{1/\alpha_B} = B_e \cdot C_{min}^{p_B}$ | $p_B = 0.24$   | $B_e = 2.0 \times 10^4$ tokens |
| $S_{min} = S_e \cdot C_{min}^{p_S}$ (lower bound)                 | $p_S = 0.03$   | $S_e = 5.4 \times 10^3$ steps  |
| $D_{opt} = D_e \cdot C_{min}^{p_D}$ (1 epoch)                     | $p_D = 0.27$   | $D_e = 2 \times 10^4$ tokens   |

Table 4 — Trends for compute-efficient training.

■ 4 — ■■■■■■■■■■

## Appendix B. Empirical Model of the Compute-Efficient Frontier



## 附录 B. 算力高效前沿的经验模型

In this appendix we derive the optimal performance, model size, and number of training steps as a function of the compute budget, starting from  $L(N, S) = (N_c / N)^{\alpha_N} + (S_c / S)^{\alpha_S}$ . Here  $S$  represents the number of parameter updates when training at the critical batch size  $B(L) = B_* / L^{1/\alpha_B}$ . We would like to determine optimal training parameters for a fixed compute budget, so we replace  $S = C / (6 N B(L))$ , where  $C$  is the number of FLOPs used in the training run. Setting  $\partial_N L|_C = 0$  yields the condition for optimality:

本附录中,我们从  $L(N, S) = (N_c / N)^{\alpha_N} + (S_c / S)^{\alpha_S}$

出发,推导损失、模型规模、训练步数随算力预算的最优变化关系。其中  $S$  是在临界批大小  $B(L) = B_* / L^{1/\alpha_B}$  下训练时的参数更新次数。为确定固定算力预算下的最优训练参数,我们将  $S$  替换为  $C / (6 N B(L))$ ,其中  $C$  是该训练运行所消耗的 FLOPs 数。令  $\partial_N L|_C = 0$ ,得到最优性条件:

$$(\alpha_N / \alpha_S) \cdot (N_c / N)^{\alpha_N} = (6 B_* S_c \cdot N / (L^{1/\alpha_B} C))^{\alpha_S}$$

$$(\alpha_N / \alpha_S) \cdot (N_c / N)^{\alpha_N} = (6 B_* S_c \cdot N / (L^{1/\alpha_B} C))^{\alpha_S}$$

Combining these equations, for compute-efficient training we should train to a fixed percentage  $\alpha_N / \alpha_S \approx 10\%$  above the converged loss. Eliminating  $N$  yields a power-law dependence of performance on compute:  $L(C) = (C_c / C)^{\alpha_C}$ , where  $\alpha_C = 1 / (1/\alpha_S + 1/\alpha_B + 1/\alpha_N) \approx 0.052$  and  $C_c = 6 N_c B_* S_c (1 + \alpha_N / \alpha_S)^{1/\alpha_S + 1/\alpha_N} \cdot (\alpha_S / \alpha_N)^{1/\alpha_S}$ . Similarly, eliminating  $L$  yields  $N(C) / N_c = (C / C_c)^{\alpha_C / \alpha_N} \cdot (1 + \alpha_N / \alpha_S)^{1/\alpha_N}$ , and  $S(C) = (C_c / (6 N_c B_*)) \cdot (1 + \alpha_N / \alpha_S)^{-1/\alpha_N} \cdot (C / C_c)^{\alpha_C / \alpha_S}$ .

结合这些方程可知:在算力高效训练下,我们应当训练至固定比收敛损失高  $\alpha_N / \alpha_S \approx 10\%$  的水平。消去  $N$ ,得到性能关于算力的幂律依赖: $L(C) = (C_c / C)^{\alpha_C}$ ,其中  $\alpha_C = 1 / (1/\alpha_S + 1/\alpha_B + 1/\alpha_N) \approx 0.052$ ,且  $C_c = 6 N_c B_* S_c \cdot (1 + \alpha_N / \alpha_S)^{1/\alpha_S + 1/\alpha_N} \cdot (\alpha_S / \alpha_N)^{1/\alpha_S}$ 。类似地,消去  $L$  得到  $N(C) / N_c = (C / C_c)^{\alpha_C / \alpha_N} \cdot (1 + \alpha_N / \alpha_S)^{1/\alpha_N}$ ,以及  $S(C) = (C_c / (6 N_c B_*)) \cdot (1 + \alpha_N / \alpha_S)^{-1/\alpha_N} \cdot (C / C_c)^{\alpha_C / \alpha_S}$ 。

Typically, researchers train models until they appear to be close to convergence. We compare the efficient training procedure to this more typical setup by defining a convergence factor  $f$  via  $L(N, C) = (1 + f) \cdot L(N, \infty)$ . For compute-efficient training we have  $f = \alpha_N / \alpha_S \approx 10\%$ , but researchers typically use a much smaller value, say  $f' = 2\%$ . For a fixed value of the loss, we predict  $N_f / N_{f'} \approx 2.7$ ,  $S_f / S_{f'} \approx 0.13$ ,  $C_f / C_{f'} \approx 0.35$ : compute-efficient training uses  $7.7\times$  fewer parameter updates,  $2.7\times$  more parameters, and  $65\%$  less compute to reach the same loss.

通常,研究者会将模型训练至接近收敛。我们通过定义收敛因子  $f$  来比较算力高效训练与

这一更通常的做法: $L(N, C) = (1 + f) \cdot L(N, \infty)$ 。对算力高效训练, $f = \alpha_N / \alpha_S \approx$

$10\%$ ;而研究者通常使用更小的值,例如  $f' = 2\%$ 。在固定损失值下,我们预测  $N_f / N_{f'} \approx 2.7, S_f / S_{f'} \approx 0.13, C_f / C_{f'} \approx 0.35$ :算力高效训练达到同一损失,使用的参数更新次数少  $7.7$  倍,参数量多  $2.7$  倍,而算力节省  $65\%$ 。

## Appendix C. Caveats

### 附录 C. 注意事项

We list some potential caveats to our analysis:

我们在本节列出对本文分析的一些潜在注意事项:

- At present we do not have a solid theoretical understanding for any of our proposed scaling laws. The scaling relations with model size and compute are especially mysterious. It may be possible to understand scaling at very large  $D$  holding model size fixed, and also the shape of learning curves late in training, by modeling the loss with a noisy quadratic. But the scaling with  $D$  at very large model size still remains mysterious. Without a theory or a systematic understanding of the corrections to our scaling laws, it's difficult to determine in what circumstances they can be trusted.

- 目前,我们对所提出的任意一条尺度定律都没有坚实的理论理解。与模型规模、算力相关的尺度关系尤其令人费解。在固定模型规模的情况下,通过用带噪二次模型来建模损失,或许可以理解极大  $D$  处的尺度以及训练后期的学习曲线形状。但在极大模型规模下, $D$  的尺度关系仍然神秘莫测。缺乏理论或对尺度定律修正项的系统理解,就很难判断其在何种情况下仍可被信赖。

- We are not especially confident in the prediction of  $B_{crit}(L)$  for values of the loss far outside the range we have explored. Changes in  $B_{crit}$  could have a significant impact on trade-offs between data parallelism and the number of serial training steps required, which would have a major impact on training time.
  - 对于远在我们所考察的损失范围之外的情况,我们对  $B_{crit}(L)$  的预测并不十分有把握。 $B_{crit}$  的变化可能会显著影响数据并行与所需串行步数之间的权衡,从而对训练时间产生重大影响。
- We did not thoroughly investigate the small data regime, and our fits for  $L(N, D)$  were poor for the smallest values of  $D$  (where an epoch corresponded to only 40 steps). Furthermore, we did not experiment with regularization and data augmentation. Improvements in these could alter our results, quantitatively or qualitatively.
  - 我们并未深入考察小数据模式,在  $D$  取最小值时(此时一个 epoch 仅对应 40 步), $L(N, D)$  的拟合较差。此外,我们也未尝试正则化与数据增强方面的实验。这些方面的改进可能在定量乃至定性上改变我们的结果。
- We used the estimated training compute  $C \approx 6 N B S$ , which did not include contributions proportional to  $n_{ctx}$  (see Section 2.1). So our scalings with compute may be confounded in practice in the regime of very large  $n_{ctx}$ , specifically where  $n_{ctx} \gg 12 d_{model}$ .
  - 我们使用的是训练算力的估计  $C \approx 6 N B S$ ,其中未包含与  $n_{ctx}$  成正比的项(参见第 2.1 节)。因此,在  $n_{ctx}$  非常大的情形(特别是  $n_{ctx} \gg 12 d_{model}$  时)中,我们对算力的尺度关系在实践中可能受到混淆。
- We tuned learning rates, and we experimented with learning rate schedules. But we may have neglected to tune some hyperparameter (e.g. initialization scale or momentum) that has an important effect on scaling.
  - 我们对学习率进行了调参,并尝试了多种学习率调度。但我们可能忽略了某些对尺度关系有重要影响的超参数(如初始化尺度或动量)的调参。
- The optimal choice of learning rate is sensitive to the target loss. When training close to convergence, it may be necessary to use a smaller learning rate to avoid divergences. But when conducting a short training run (e.g. due to compute limitations), it may be possible to use a larger learning rate. We did not experiment with higher learning rates for training runs that did not proceed to convergence.
  - 最优学习率的选择对目标损失敏感。当训练接近收敛时,可能需要使用更小的学习率以避免发散;而当进行短时间训练(例如受算力限制)时,则可以使用更大的学习率。我们并未对那些未训练至收敛的运行尝试更高的学习率。

*Note: This is a faithful, full-text translation of the main body of the paper, plus the key appendix tables (Tables 2–4 of the Appendix). Figures have been referenced by number but not re-rendered; please consult the original paper for figures. Mathematical notation is linearized for PDF rendering (e.g.,  $C_{min}$ ,  $\alpha_N$ ,  $D \sim C^{0.27}$ ).*